Route Optimization For Emergency Vehicles

A PROJECT REPORT

Submitted by

THILLAI RAJ	**19BEC086**
AAKAASH N	**19BEC103**
MINISH S	**19BEC097**

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING

KUMARAGURU COLLEGE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

COIMBATORE – 641049

DECEMBER 2022.

BONAFIDE CERTIFICATE

Certified that this project report **"ROUTE OPTIMIZATION FOR EMERGENCY VEHICLE"** is the Bonafede work of

THILLAI RAJ **(19BEC086)**
AAKAASH N **(19BEC103)**
MINISH S **(19BEC097)**

Who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
Dr. S.A. PASUPATHY, Ph.D. (UK).	**Dr. S. DAVID, Ph.D.**
	(Assistant Professor II)
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Dept of Electronics & Communication Engineering, Kumaraguru College of Technology. Coimbatore – 641049.	Dept of Electronics & Communication Engineering, Kumaraguru College of Technology. Coimbatore – 641049.

The candidates were examined by us in the project viva voce examination held on …….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We express our sincere thanks to the Management of Kumaraguru College of Technology and **Shri. Shankar Vanavarayar**, Joint Correspondent, Kumaraguru Institutions for the kind support and for providing necessary facilities to carry out the project work.

We would like to express our sincere thanks to our beloved **Principal Dr. D Saravanan, Ph.D.**, Kumaraguru College of Technology, who encouraged each and every step of the project.

We would like to thank **Dr.S.A.Pasupathy, Ph.D (UK)**, Head of the Department, Electronics and Communication Engineering, for his kind support and for providing necessary facilities to carry out the project work.

We wish to thank with everlasting gratitude to our Project Coordinator **Mr. Boopathy S,** Assistant Professor I, Department of Electronics and Communication Engineering for his consistent support throughout the course of this project work.

We are greatly privileged to express our deep sense of gratitude and heartfelt thanks to our Project Guide **Dr. S David, Ph.D,** Assistant Professor II, Department of Electronics and Communication Engineering for his expert counselling and guidance to make this project to a great deal of success and also we wish to convey our regards to all teaching and non-teaching staff of ECE Department for their help and cooperation.

Finally, we thank our parents and our family members for giving us the moral support and abundant blessings in all of our activities and our dear friends who helped us to endure our difficult times with their unfailing support and warm wishes.

ABSTRACT

In this modern era getting struck in busy traffic for hours makes human impatient. Movement of emergency vehicles among these busy routes has become more tougher. So there is an immediate need for an optimized traffic flow control. This work delivers the study of traffic flow analysis in busy areas . Implementation of a suitable algorithm to analyse quick route to reach the destination for emergency vehicle is being analysed here. This method is based on graph theory, where the data in the data sheet is first converted into graph with nodes and edges. Here, nodes represent the area and the edge represents the time required to travel through. The edges are given with different weight i.e. the distance and the traffic percentage in that route. Based on the value of these data the total time required to travel through the route is calculated.

Based on this calculated value the shortest path is obtained by using the dijkstra's algorithm.

APPENDIX I

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	TITLE PAGE	**1**
	BONAFIDE CERTIFICATE	**2**
	ACKNOWLEDGEMENT	**3**
	ABSTRACT	**4**
	TABLE OF CONENT	**5**
	LIST OF FIGURES	**8**
1	**INTRODUCTION**	**9**
	1.1 PROBLEM STATEMENT	9
	1.2 IMPORTANCE OF THE PROPOSED IDEA	9
	1.3 IMPACT	10
	1.4 COMPETITION	10
	1.5 CHALLENGES	10
2	**LITERARURE SURVEY**	**11**

3 FEEDBACK SURVEY 15

4 PROPOSED SYSTEM 19

4.1 IDEA/CONCEPT GENERATION 19

4.2 REASON FOR FINALISATION 20

5 SOFTWARE DESCRIPTION 21

5.1 PYTHON 21

5.1.1 DESCRIPTION 21

5.1.2 FUTURE TECHNOLOGY ON PYTHON 22

5.2 ANACONDA 23

5.2.1 DESCRIPTION 23

5.2.2 SWITCHING CONDA ENVIRONMENT 23

5.3 JUPITER NOTEBOOK 24

6 WORKING (DESIGN) 25

6.1 METHODOLOGY 25

6.2 FLOW CHART 26

6.3 DESIGN 27

7	**SIMULATION**	**28**
	7.1 TOOLS	28
	7.2 SIMULATION	28
	7.3 ALGORITHM	29
	7.3.1 GRAPH ALGORITHM	29
	7.3.2 DIJKSTRA'S ALORITHM	29
	7.3.3 TIME ALGORITHM	31
8	**RESULT AND DISCUSSION**	**33**
	8.1 SAMPLE DATA SET	33
	8.2 CALCULATION	33
	8.3 RESULT	34
	8.3.1 INPUT	34
	8.3.2 OUTPUT	34
	8.4 EXISTING TO MODIFIED	36
	8.5 DISCUSSION	36
	8.6 CONCLUSION	37
	REFERENCE	**38**

APPENDIX II

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	Number of responses	**15**
3.2	Query 1	**15**
3.3	Query 2	**16**
3.4	Query 3	**16**
3.5	Query 4	**17**
3.6	Query 5	**18**
6.1	Flowchart of Traffic Routing algorithm	**26**
7.1	Dijkstra's Algorithm	**31**
7.2	Graph and Time Algorithm	**32**
8.1	Data Set	**33**
8.2	Output of Time Algorithm	**35**
8.3	Output of Final Algorithm	**35**
8.4	Comparison	**36**

CHAPTER 1

INTRODUCTION

1.1 PROBLEM STATEMENT

The objective of this work is to predict the traffic flow in urban areas and to provide an efficient way for movement of emergency vehicles to reach their destination on time. Due to heavy traffic congestion, most of the patients who are travelling on an emergency vehicles like ambulance, get struck in the traffic which leads to delay in reaching the hospital on time. Because of this reason, many patients condition becomes severe and sometimes leads to death. In order to prevent this critical situation an solution has been deployed to detect and prevent the traffic flow specifically for emergency vehicles.

1.2 IMPORTANCE OF THE PROPOSED IDEA

Approximately 60% of the people get struck in the traffic in their day to day life which results in major problems. Transport sector is important for the economic growth of our nation. Because of these delays in transportation sector it leads to some critical situation that affect the business of many private and public sectors. The delay and loss of a business sector is very mere when compared to the delay in an emergency vehicle as it completely deals with human lives. Even a small delay will lead to critical situation of a persons, sometimes may also leads to death.

1.3 IMPACT

To introduce an algorithm that preciously finds the shortest path and guide the emergency vehicle to reach the destination at faster rate.

The solution helps not only emergency vehicle but also the common people who wishes to reach their destination on time. As a result, no one can have to wait for a long time in the traffic by finding the better ways to reach their destination.

1.4 COMPETITION

The dijkstra's algorithm used here finds the shortest path based on the distance between the two areas through different routes. This algorithm is used in many company such as google maps, apple maps, android auto etc. The main drawback of this algorithm is that it cannot find the traffic analysis of any particular area. All nodes that are still immobile are considered to a load. It is important to detect and analyze the traffic flow in that particular area and find the shortest and fastest route to reach the destination for all moving nodes.

1.5 CHALLENGES

- Instead of finding the distance between the two region, calculating the time required to reach the destination is an difficult process, also the actual time taken can vary from different time basis.

- Continuous monitoring of traffic in all areas is also a difficult process in developing this system.
- Fetching the time information from a particular area from a third party source is also more expensive.
- Implementing the suggested algorithm to educate the general public may be a challenge due to the complexity of the process.

CHAPTER 2

LITERATURE SURVEY

1. **traffic flow prediction using machine learning and deep learning. Journal of Big Data volume 8, Article number: 152 (2021), Dec 04, 2021.**

In recent years, there has been increased interest in the use of machine learning (ML) and deep learning (DL) in order to improve traffic flow forecasting and prediction. This literature review explores the application of ML and DL to the Internet of Things (IoT) and Intelligent Transportation Systems (ITS). It examines how ML and DL can be used to improve existing features, as well as to bridge the gap between the system of IoT and ITS, which may result in improved performance. Additionally, it is shown that ML and DL can improve algorithms, and that local urban authorities can benefit from their application.

2. **IoT security with Deep Learning-based Intrusion Detection Systems. 4th ICDS , 21-23, October 2020.**

The IoT security deep learning based intrusion detection system has a major role in modern times. This system has been developed to help protect against security threats and researchers are looking into ways to integrate the system into real network traffic to help detect cyber attacks. New datasets are also being introduced to the IOT environment in order to reduce false positives and negatives. Algorithms are being developed and tested to give the intrusion detection system a powerful upgrade.

3. **IoT-based traffic prediction and traffic signal control system for smart city. ResearchGate,19 May 2021**

The proposed traffic prediction and signal control system for smart cities utilizes the OWENN algorithm, Intel 80,286 microprocessor, and five stages of data collection in order to reduce traffic congestion that commonly occurs in metropolitan cities. This system was compared with existing ENN, CNN, NN and ANFIS techniques, using MAE, RMSE, and fmeasure metrics to measure accuracy. The results showed an accuracy of 98.23% and a measure of 96.69%. Compared to existing techniques, this system provided lower efficiency, but uses Wi-Fi communication to improve energy usage and charging solutions for the future.

4. **KCLP: A k-Means Cluster-Based Location Privacy Protection Scheme in WSNs for IoT. IEEE Wireless Communications, December 2018.**

KCLP is a K-Means Cluster-Based Location Privacy Protection Scheme in WSNs for loT, designed to reduce the problems associated with wireless sensor networks. This scheme creates a fake source, which functions similarly to a real source, and a fake sink, which also performs the same role. The KCLP increases safety time and reduces delay while consuming less energy. It has been compared to SLP and RBR, with results indicating that it is more efficient than both.

5. **An LSTM-Based Deep Learning Approach for Classifying Malicious Traffic at the Packet Level. IEEE ICASI , 19 August 2019.**

This paper proposes a novel approach utilizing an LSTM model to classify malicious traffic at the packet level, with the aim of boosting the detection process of malicious attacks. It is competitive to prior literature that detects malicious traffic on the flow level and is inspired by the increasing prevalence of malicious attacks year by year. It offers the advantage of not requiring preprocessed packets to create flows and thus reduces the delay of the intrusion detection system (IDS).

CHAPTER 3

FEEDBACK SURVEY

This is a society issue so we took a survey to develop our project in a better way. Many people have felt bad to get struck in the traffic but reasons will be differ by person to person. At last part of our survey we asked for a suggestion that our product will be useful or not for the society. We got a positive result that many people accepted our idea.

Responses:

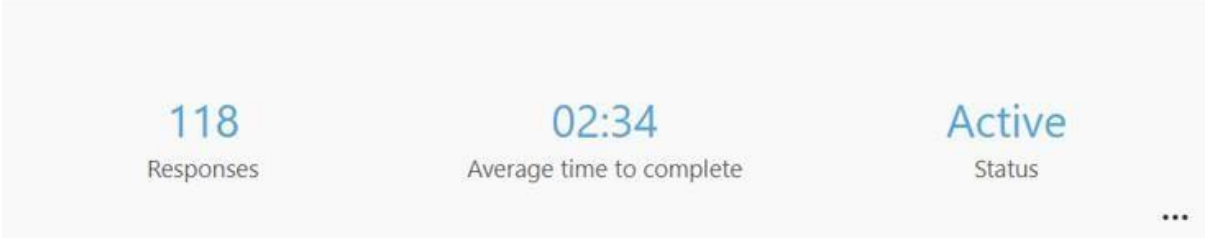


Figure 3.1 – **Number of responses**

Survey:

1. Have you ever struck in traffic

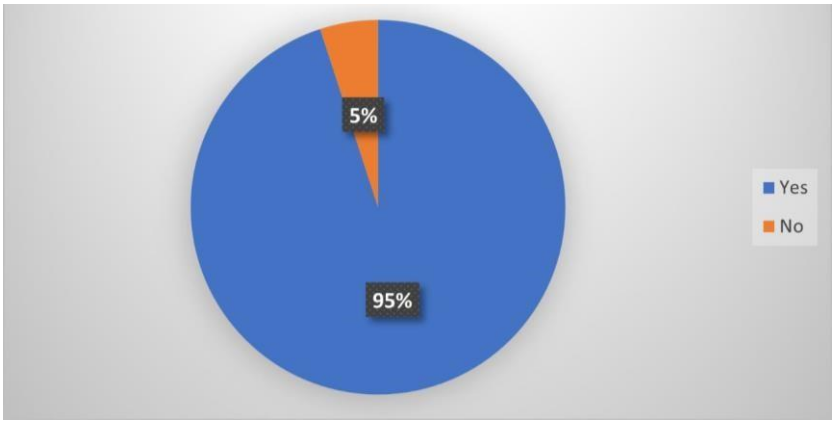


Figure 3.2– **Query 1**

It is obvious that, on their journey to their destination, 95% of people get hit by traffic. Many of them are unable to arrive to work on time as a result of this issue.

2. Are you Employee or Senior citizen

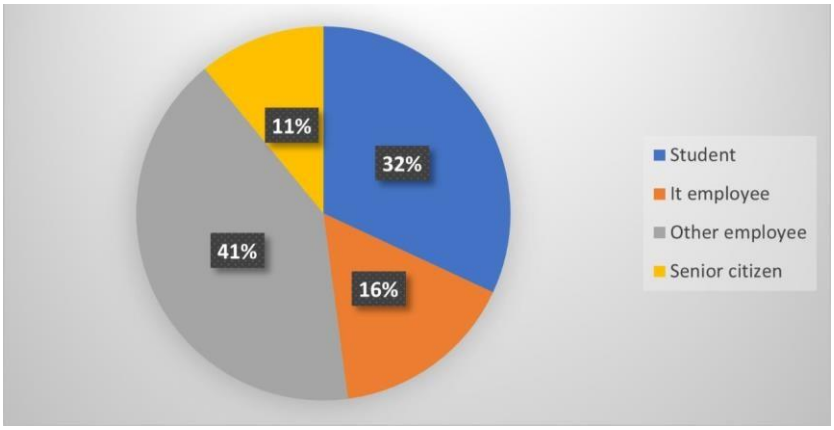


Figure 3.3 – **Query 2**

It is obvious that, on their journey to their destination, 95% of people get hit by traffic. Many of them are unable to arrive to work on time as a result of this issue.

3. How many times have you struck in the traffic

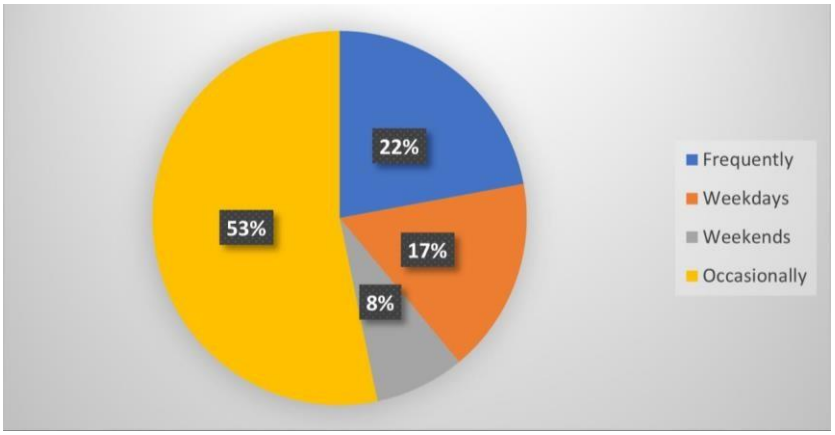


Figure 3.4 – **Query 3**

Our survey clearly shows that the majority of them routinely encounter traffic on their approach to their destination. It has an impact on many people's everyday routines at work.

4. Do you feel that the present system gives you solution for the traffic flow

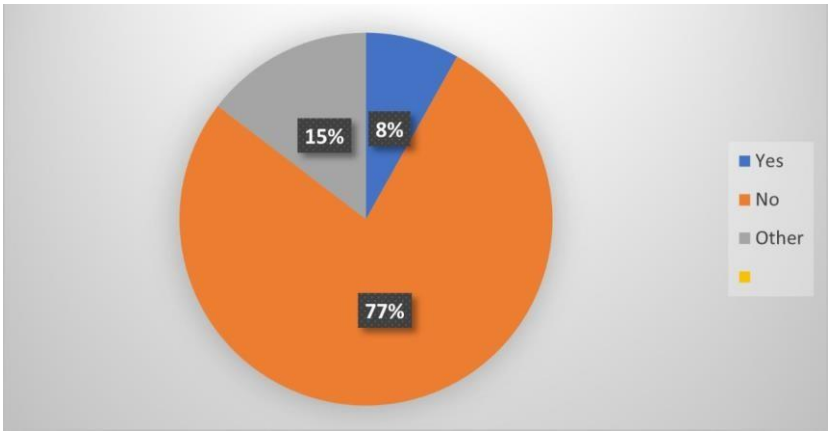


Figure 3.5 – **Query 4**

Our poll clearly shows that the majority of those who are hit in traffic cannot be satisfied by the current mechanism to control traffic flow. For those folks, we need a different approach.

5. Which algorithm have you ever heard for traffic flow control

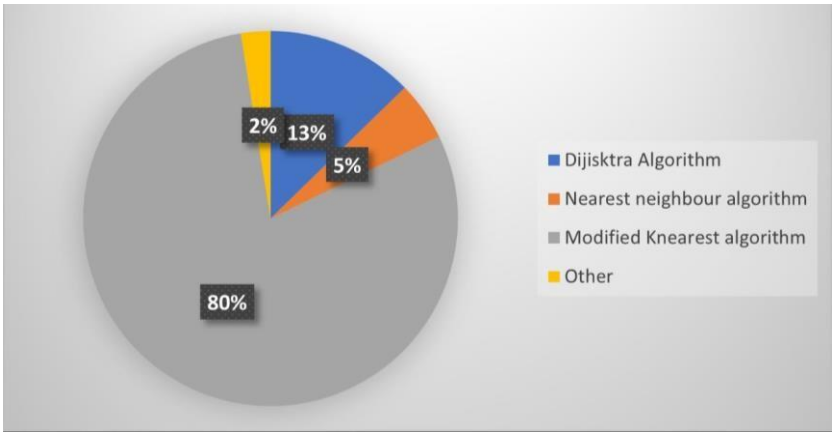


Figure 3.6 – **Query 5**

From our survey, it is evident that most of the people who get struck in the traffic knows the k nearest algorithm. some of them may also knows about the dijkstra's algorithm.

CHAPTER 4

PROPOSED SYSTEM

4.1 IDEA/CONCEPT GENERATION

The system is used to develop the section of a road network where the user gives the input as the origin and the destination. Thus the system applies the dijkstra's algorithm to give the output as the fastest route the approximate time to reach the destination. The output results in each and every area that has been covered to travel from origin to destination along with their respective distance and time required to cross that corresponding intermediate areas. e, the algorithm considers the route with the minimum time and returns it together with the total approximate time in minutes.

All the areas in that particular shortest route is calculated and shown in the output. This helps us to keep track the areas and reach the destination by easily identifying the routes and areas. This system helps not only the regular vehicle but also the emergency vehicle to reach the hospital very soon and helps the patient to save their lives.

4.2 REASON FOR FINALIZATION OF CONCEPT

The main reason to final this concept is to detect the traffic flow control in an efficient way by calculating the total time required to travel from one place to another rather than finding the distance between the two places. It is more efficient then the existing Dijkstra's algorithm by producing the accurate result with the calculation of total time required to cover the required distance between the two places. This system finds the shortest path on the basis of both the distance between the two nodes and also the traffic congestion in that particular areas.

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 PYTHON

5.1.1 DESCRIPTION:

Python is a programming language that allows you to work quickly and integrate systems more efficiently. Python is a high-level, interpreted, interactive, and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords often where other languages use punctuation, and has fewer syntactic constructions than other languages.

Python is Interpreted − Python is processed at runtime by an interpreter. You don't need to compile before running the program. It is similar to PERL and PHP. Python is interactive − You can actually sit at the Python command line and interact with the interpreter right as you write your programs. Python is object-oriented – Python supports an object-oriented style or programming technique that encapsulates code in objects**.**

- **Easy to learn** − Python has few keywords, a simple structure, and a clearly defined syntax. This allows the student to acquire the language quickly.
- **Easy to read** − Python code is more clearly defined and visible to the eye.
- **Ease of Maintenance** − Python source code is relatively easy to maintain.

- **Extensive standard library** - Much of Python's library is highly portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive mode** − Python has support for an interactive mode that allows for interactive testing and debugging of code snippets.
- **Portable** − Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extensible** − You can add low-level modules to the Python interpreter. These modules allow programmers to add or customize their tools to make them more efficient.
- **Database** − Python provides an interface to all major commercial databases.
- **GUI Programming** - Python supports GUI applications that can be built and ported to many system calls, libraries, and Windows systems such as Windows MFC, Macintosh, and Unix's X Window System.
- **Scalability** − Python provides better structure and support for large programs than shell scripting.

5.1.2 FUTURE TECHNOLOGY COMPUTING PYTHON:

Generally, we have seen that python programming language is widely used for web development, application development, system administration, game development, etc.

However, there are some future technologies that rely on python. In fact, Python has become an essential language when it comes to the success of

these technologies. Let's dive into the technologies that use python as a core element for research, production and further development.

5.2 ANACONDA

5.2.1 DESCRIPTION:

Anaconda is a great package that already contains many Python packages, making it easy to enter the Python world. Additionally, you can create environments in Python that contain different versions of Python packages. For example, if your program only runs on versions of Matplotlib prior to Python 2.7, you can create a separate workspace for this program and switch back to Python 3 with one click. In addition, switching between Tensorflow 2.0 and Tensorflow 1.15 will also be easier, so you will eventually be able to switch between versions without any problems (otherwise it can cause quite a headache).

Miniconda is a barebones version of Anaconda. Working on servers with limited disk space. When Anaconda (conda) and Jupyter Notebook (Jupyter Lab) are set up properly, their combination makes for a perfect team that can easily switch between deep learning conda environments.

5.2.2 SWITCHING CONDA ENVIRONMENT:

- If conda is installed correctly (you may need to logoff and logon or reboot), type "conda" in the terminal and you should see the output.
- To list the currently installed environments, type "conda env list".
- Currently only "Basic" installations are shown.

- Switching between environments is as easy as typing "conda activate [NAME]". Once done, deactivate (and return to the base environment) with conda activate.
- Base environment is activated by default.

5.3 JUPITER NOTEBOOK

- Jupyter Notebook is an open source web application that lets you create and share documents with live code, equations, visualizations, and text. The Jupyter Notebook is maintained by Project Jupyter staff.
- Jupyter Notebooks is a project derived from his IPython project, where previously he had an IPython notebook project itself. The name Jupyter comes from core programming languages that support Julia, Python, and R. Jupyter comes with an IPython kernel that allows you to program in Python, but there are currently over 100 other kernels available. as well.
- Jupyter Notebooks are very useful not only for learning and teaching programming languages like Python, but also for sharing data.
- Turn your notebooks into slideshows or share them online on GitHub. If you want to share notebooks without users having to install anything, you can use Binder to do so.
- Google and Microsoft have their own versions of his Notebook that can be used to create and share notebooks on Google Colaboratory and Microsoft Azure Notebooks respectively. You can also browse some very interesting notebooks.
- Project Jupyter recently launched its latest product, JupyterLab. JupyterLab integrates Jupyter Notebook with an integrated development type editor that runs in your browser. JupyterLab can be thought of as an enhanced version of Jupyter Notebook.

CHAPTER 6

WORKING (DESIGN)

6.1 METHODOLOGY

The design of our research requires the real time complete data set and conversion of this data set into the graph with nodes and edges. The data set must consist of the necessary column that has been implemented in the algorithm. The data set must contain the source information, destination information, their corresponding latitude and longitude and their distance between the source and the destination and their corresponding traffic percentage compared to standard rates. With these values on the data set, it is first converted into corresponding graph with nodes and edges. The nodes represent the area of source and destination and the edge represents the time required to reach the destination through the particular route.

Once the graph is ready the dijktra's algorithm is used to determine the shortest path between the source and the destination.

.

6.2 FLOW CHART

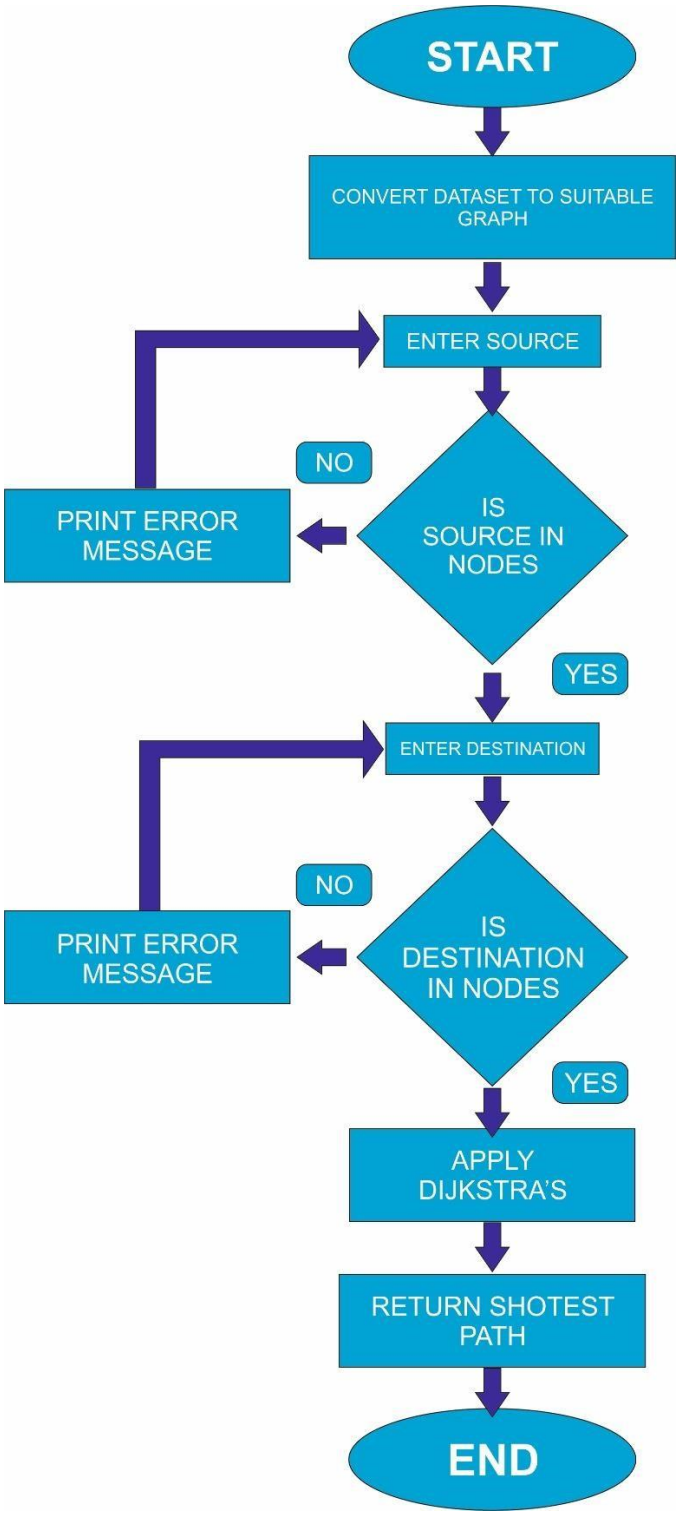


Figure 6.1 – **Flowchart of traffic routing algorithm.**

6.3 DESIGN:

A section of the data set shown in the figure is converted into the graph. This section of roads is chosen for the purposes of demonstrating the working of the algorithm. This graph is used for the detection of shortest path for transportation. This data set contains the information about the distance and weightage of the traffic percentage in that particular route. After the formation of graph, the user have to enter the source and the destination place. The algorithm will check the availability of that particular place in the graph. If there is no place available in that graph, then the algorithm will through an error indicating that there is place in that data set. After recognization of the source and destination the code will enter into the dijkstra's algorithm to find the shortest path. The algorithm will take the graph, source and destination as an input and will find out the shortest path to reach the destination from the source.

CHAPTER 7

SIMULATION

7.1 TOOLS

This paper primarily uses the Dijkstra's algorithm to determine the shortest path between the source and the destination. The algorithm is implemented using python language. Therefore, all the program writing is done using python. Using the concept of graph theory, the road network between the source and the destination is studied thoroughly and analysed. Thereby helps to detecting the shortest path between the source and the destination by using the graph concept.

7.2 SIMULATION

To simulate the working of the algorithm, a detailed data about the road network is first converted into the graph. The time taken to move from one location to another is obtained from the calculation of the distance and weightage of the traffic in that route along with the standard average speed calculation. Dijkstra's algorithm is applied on the graph where one input is taken from the origin and the other from the destination and the algorithm returns the fastest route and the shortest time a vehicle is likely to take. Several tests are conducted by running the program at different times.

7.3 ALGORITHM

7.3.1 GRAPH ALGORITHM:

```
class Graph():     def _init_(self):         self.edges =
defaultdict(list)         self.weights = {}      def
add_edge(self, from_node, to_node, weight):
# Note: assumes edges are bi-directional
self.edges[from_node].append(to_node)
self.edges[to_node].append(from_node)
self.weights[(from_node, to_node)] = weight
self.weights[(to_node, from_node)] = weight
```

7.3.2 DIJKSTRA'S ALGORITHM:

```
def dijsktra(graph, initial, end):
   # shortest paths is a dict of nodes
   # whose value is a tuple of (previous node, weight)
shortest_paths = {initial: (None, 0)}      current_node
= initial      visited = set()      while current_node !=
end:         visited.add(current_node)

       destinations = graph.edges[current_node]
weight_to_current_node = shortest_paths[current_node][1]         for
next_node in destinations:
```

```
        weight = graph.weights[(current_node, next_node)] + weight_to_current_node
        if next_node not in shortest_paths:
            shortest_paths[next_node] = (current_node, weight)
        else:
            current_shortest_weight = shortest_paths[next_node][1]
if current_shortest_weight > weight:
                shortest_paths[next_node] = (current_node, weight)
next_destinations = {node: shortest_paths[node] for node in shortest_paths if node not in visited}
if not next_destinations:
        return "Route Not Possible"
    # next node is the destination with the lowest weight
    current_node = min(next_destinations, key=lambda k: next_destinations[k][1])
  # Work back through destinations in shortest path
path = []
total=0
while current_node is not None:
        path.append(current_node)
#print(current_node)
        next_node = shortest_paths[current_node][0]
        if next_node is not None:
            print("from : {} , to : {} , distance : {} km".format(next_node,current_node,graph.weights[(next_node, current_node)]))
total = total+graph.weights[(next_node, current_node)]
        current_node = next_node
```

Reverse path print("Total Distance = {}

km".format(total)) path = path[::-1] return

path

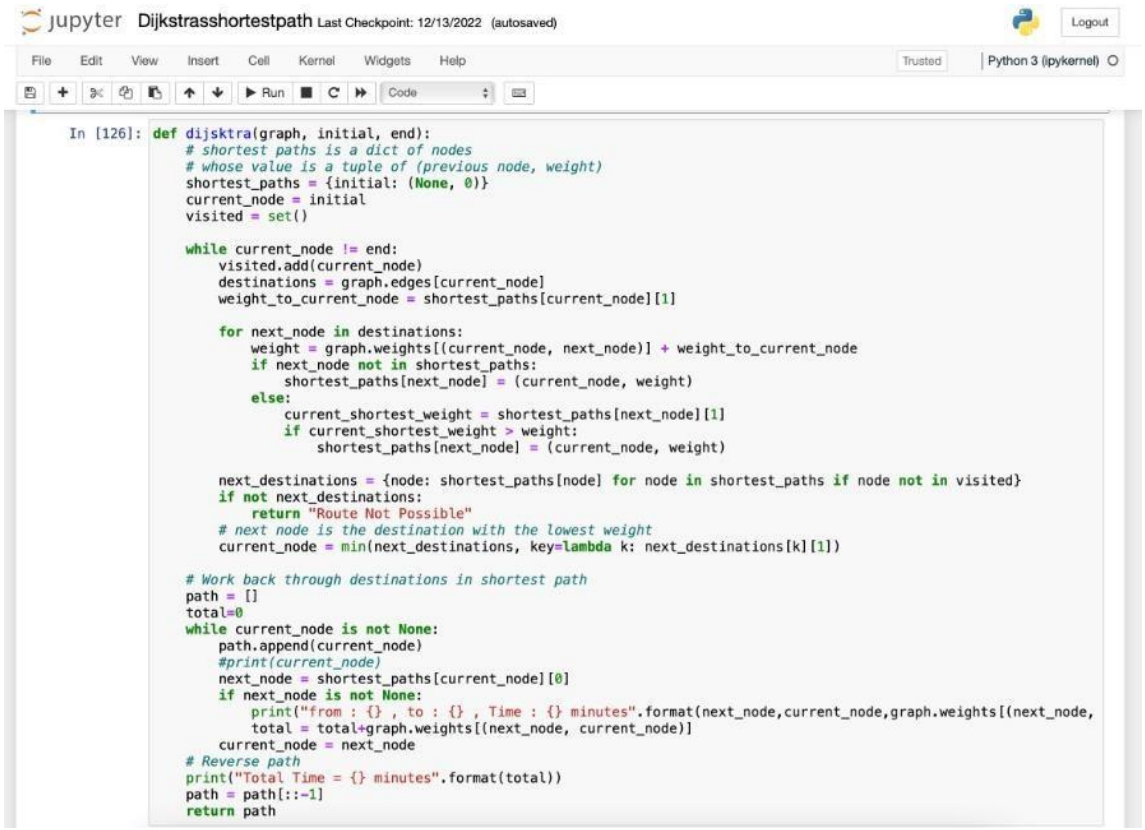


Figure 7.1 – **Dijkstra's Algorithm**

7.3.3 TIME ALGORITHM

with open('data.csv', 'r') as read_obj:

csv_dict_reader = DictReader(read_obj)

Average = 60 for row in

csv_dict_reader:

Distance = int(row['distance'])

Percentage = int(row['traffic percentage'])

Decreased_speed = Average * (Percentage/100)

Actual_speed = Average - Decreased_speed

Total_time = Distance / Actual_speed Total_time = int(Total_time * 60)

print("Source Destination Distance Average Speed Reduced speed Actual speed time")

print("{} {} {} {} {} {} {}".format(row['from'], row['to'],Distance,Average,Decreased_speed,Actual_speed,Total_time))

graph.add_edge(row['from'], row['to'], Total_time)

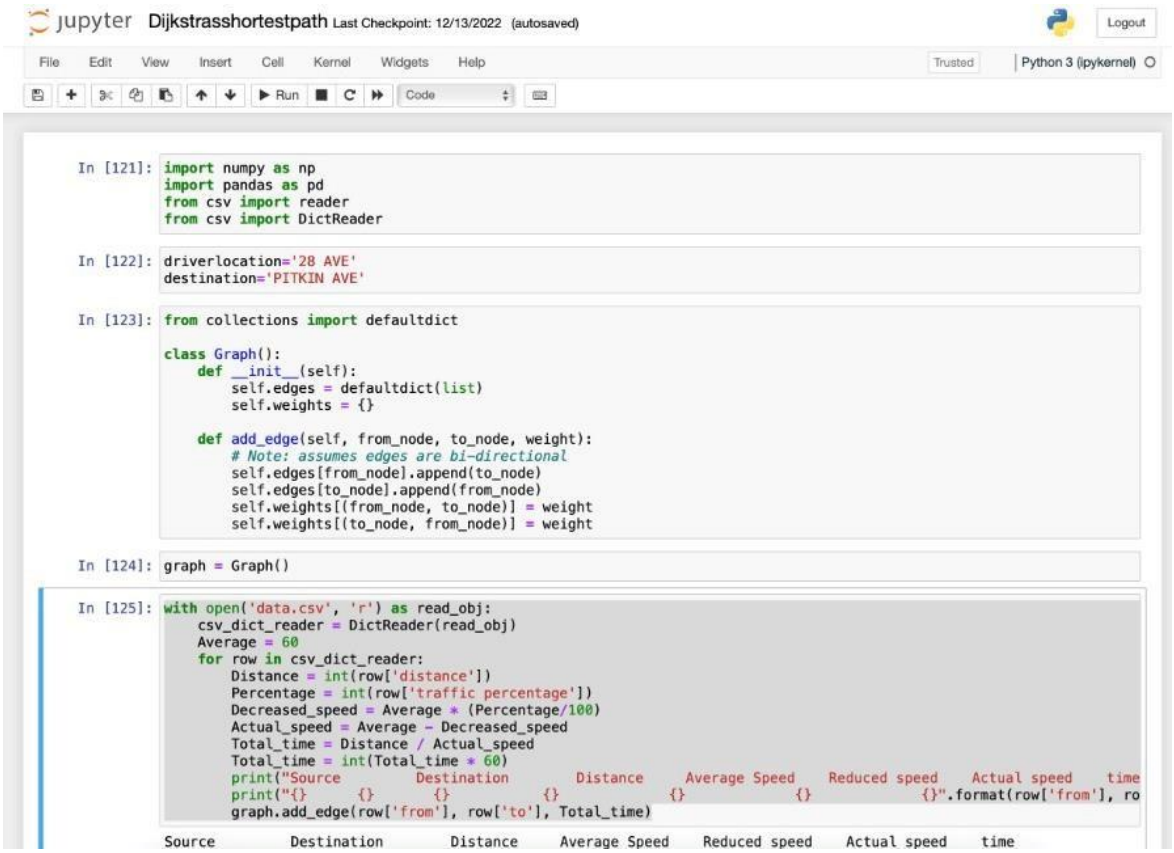


Figure 7.2 – **Graph and Time Algorithm.**

CHAPTER 8

OUTPUT AND DISCUSSIONS

8.1 SAMPLE DATA SET

Source	Destination	Distance (km)	Traffic Percentage			
28 AVE	SCHERMERHORN ST	50	15			
28 AVE	ARLINGTON AVE	40	23			
28 AVE	QUEENS BLVD	25	20			
28 AVE	WALWORTH ST	25	15			
SCHERMERHORN ST	ARLINGTON AVE	15	23			
SCHERMERHORN ST	ARTHUR KIL RD	15	26			
ARLINGTON AVE	ARTHUR KIL RD	25	30			
ARLINGTON AVE	FULTON AVE	25	28			
QUEENS BLVD	PITKIN AVE	40	31			

Figure 8.1 - **Data set**

8.2 CALCULATION

D – distance between two nodes.

P – percentage of traffic flow.

S – Average speed.

T – Time taken to reach the destination.

$$T = D / (S - ((S * P) / 100))$$

8.3 RESULT

8.3.1 INPUT:

Source: 28 AVE.

Destination: PITKIN AVE.

8.3.2 OUTPUT:

from : NORTON ST , **to** : PITKIN AVE , **Time** : 23 minutes **from** : QUEENS BLVD , **to** : NORTON ST , **Time** : 17 minutes **from** : 28 AVE , **to** : QUEENS BLVD , **Time** : 31 minutes

Total Time: 71 minutes

Source : 28 AVE

QUEENS BLVD

NORTON ST

Destination : PITKIN AVE.

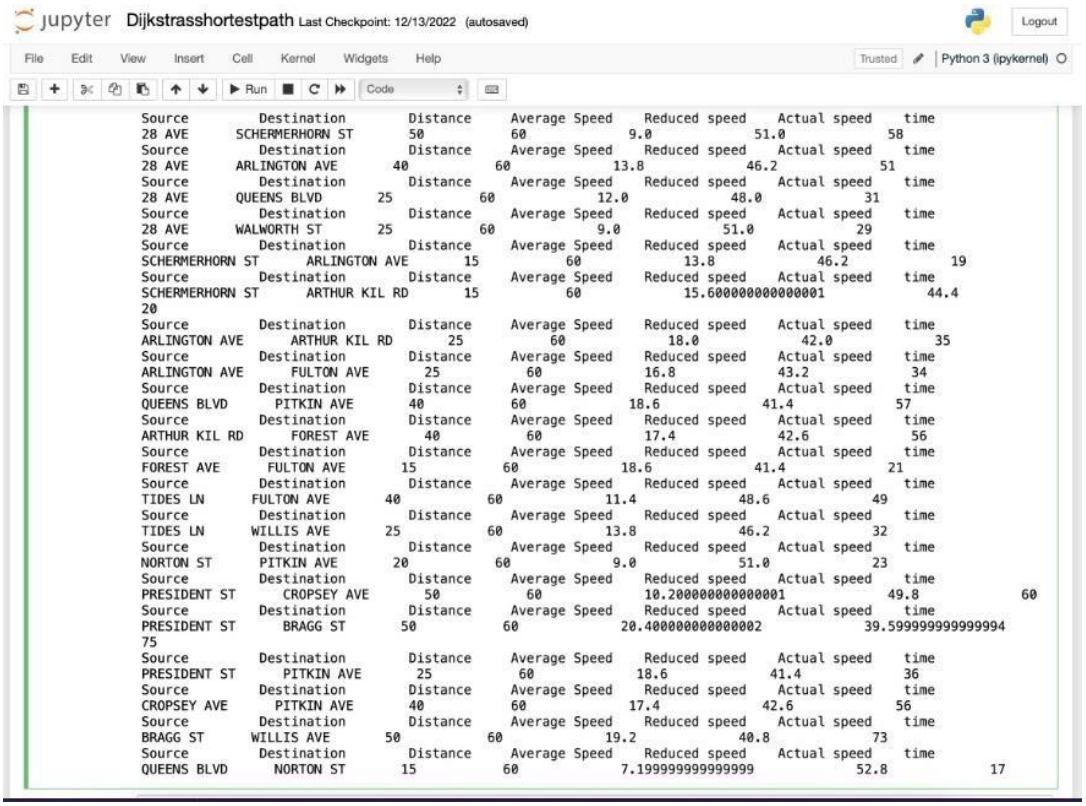


Figure 8.2 – **Result of Time Algorithm.**

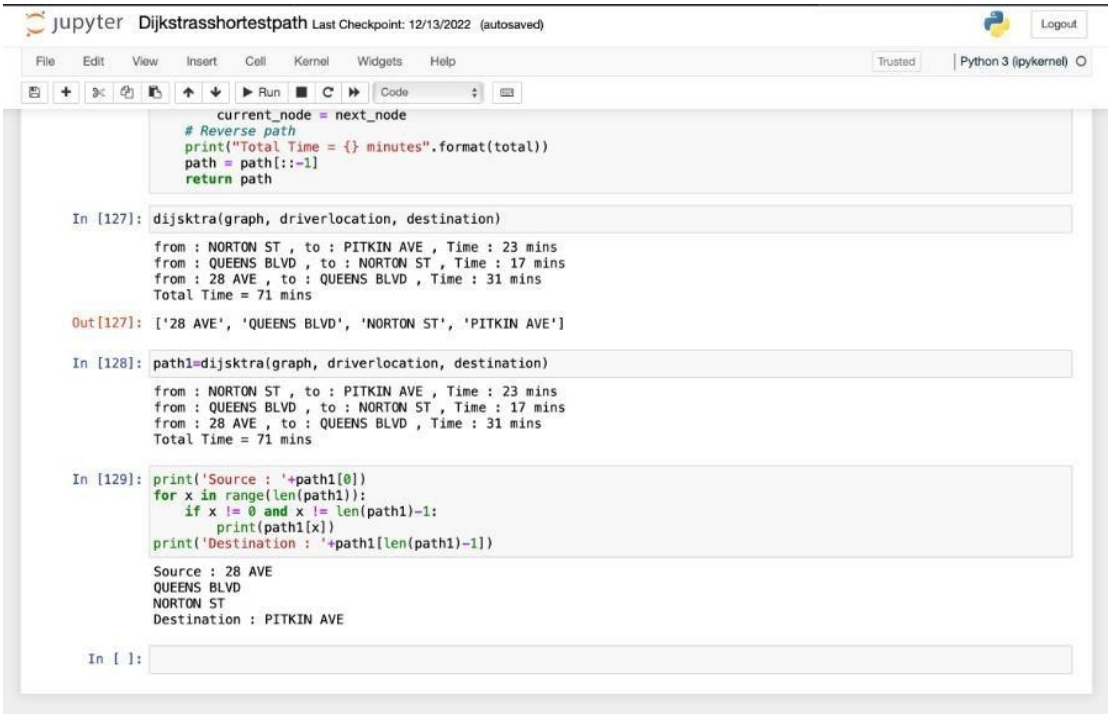


Figure 8.3 – **Output of Final Algorithm.**

8.4 EXISTING TO MODIFIED:

The existing algorithm is used to calculate the shortest path only using the distance between the source and the destination. Whereas, the modified algorithm calculate the shortest path based on the value of time taken to reach the destination based on the calculation of both the distance between the source and the destination and also with the average traffic percentage in that particular route.

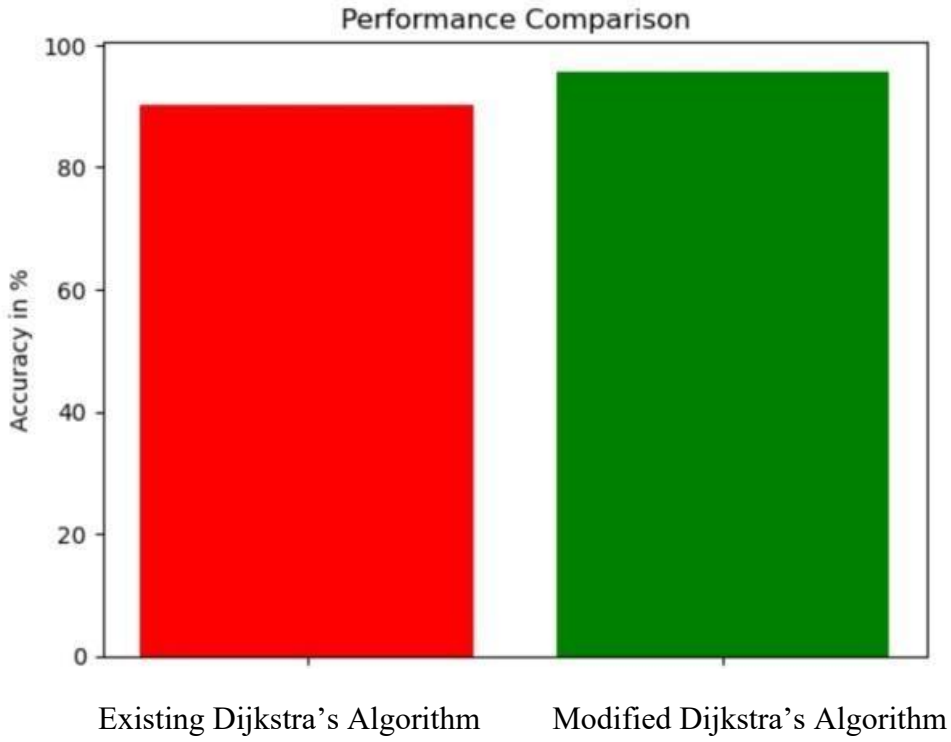


Figure 8.4 – **Comparison.**

8.5 DISCUSSION

The system is used to develop the section of a road network where the user gives the input as the origin and the destination. Thus the system applies the dijkstra's algorithm to give the output as the fastest route the approximate time to reach the destination. The output results in each and every area that has been

covered to travel from origin to destination along with their respective distance and time required to cross that corresponding intermediate areas. e, the algorithm considers the route with the minimum time and returns it together with the total approximate time in minutes.

As seen in the result, all the areas in that particular shortest route is calculated and shown in the output. This helps us to keep track the areas and reach the destination by easily identifying the routes and areas. This system helps not only the regular vehicle but also the emergency vehicle to reach the hospital very soon and helps the patient to save their lives.

8.6 CONCLUSION

In conclusion, it is evident that in the current world with developing technology, efficiency and reliability are critical when developing new systems and implement in the real world. There is a high need to save time especially on the roads during critical situation and also obtain reliable information on the traffic situation that can be relied upon. The system developed shows that the fastest route from one point to another can be determined with a suitable algorithm and also helps to save many lives by its efficient traffic flow control methodology. This meets the main objective of the paper.

REFERENCES

- *Agafonov, A., & Myasnikov, V. (2017). Efficiency comparison of the routing algorithms used in centralized traffic management systems. Procedia Engineering, Vol. 201, 265-270.*

- *Agafonov, A., & Myasnikov, V. (2018). Vehicle routing algorithms based on a route reservation approach. Journal of Physics: Conference Series, Vol. 1069, 1-9.*

- *Mu, H., Yu, J., & Liu, L. (2009). Shortest path algorithm for road network with traffic restriction. IEEE 2nd International Conference on Power Electronics and Intelligent Transportation System, China.*

- *Nha, V., Djahel, S., & Murphy, J. (2012). A comparative study of vehicles' routing algorithms for route planning in smart cities. IEEE First International Workshop on Vehicular Traffic Management for Smart Cities, Ireland.*

- *Noor Afiza Mat Razali, Nuraini Shamsaimon, Khairul Khalil Ishak. Traffic flow prediction using machine learning and deep learning. Journal of Big Data volume 8, Article number: 152 (2021), Dec 04, 2021.*

- *Idriss Idrissi, Mostafa Azizi, Omar Moussaoui. IoT security with Deep Learning-based Intrusion Detection Systems. 4th ICDS , 21-23, October 2020.*

- *S. Neelakandan, M. A. Berlin, Sandesh Tripathi, V. Brindha Devi. IoTbased traffic prediction and traffic signal control system for smart city. ResearchGate,19 May 2021 .*

- *Guangjie Han, Hao Wang, Mohsen Guizani, Sammy Chan. KCLP: A kMeans Cluster-Based Location Privacy Protection Scheme in WSNs for IoT. IEEE Wireless Communications, December 2018.*